

CSP-S / NOIP 提高组知识自查表

提高级知识、复杂度与实现检查

示例基线：C++14 (-O2 -std=c++14 -static), 以当年比赛公告为准。

使用说明：按知识点检查适用条件、正确实现及时间、空间复杂度，并用边界数据或反例验证。 **星号说明：**带 * 的内容不属于提高级大纲的知识类别，属于 NOI 级或补充内容；提高级类别下的常见实现、变体和应用不标星。

目录

1 C++ 语言与实现	2
2 基础算法与问题求解	4
3 字符串	6
4 数据结构	7
5 图论	9
6 树论	11
7 动态规划	12
8 数学	13
9 算法复杂度索引	15
10 空间复杂度与内存估算	17
11 时间复杂度估算基准	18
12 *C++ 未定义行为与实现风险	19
13 * 对拍与差分测试	21
14 *NP、NP 完全与 NP-hard 识别	22
15 能力缺口与复习重点	23
创作声明与许可	23

1 C++ 语言与实现

知识点	适用范围与要点	掌握要求
整数类型范围与溢出	乘法、平方及累计值可能超过 $2^{31} - 1$ 时需要扩大类型。两个 <code>int</code> 的乘法在赋值给 <code>long long</code> 之前已经按 <code>int</code> 计算；比例乘积、平方和及组合数递推均需检查中间值。	写出安全乘法 <code>bool mul_ok(ll a, ll b, ll limit)</code> ，用除法预判而非先乘。
整数除法、取模与模运算陷阱	整除、向上取整 $(a+b-1)/b$ 、负数取模符号（C++14 向零取整）。模运算减法必须 $(a-b+MOD)\%MOD$ ；取模前的乘法必须用 $1LL*a*b\%MOD$ 。	实现分数加法输出最简分数，用 <code>gcd</code> 约分避免中间溢出；在模 $10^9 + 7$ 下求组合数，区分质数模（可用阶乘逆元）和非质数模（必须用杨辉三角取模递推）。
浮点误差与精确比较	概率期望 DP 中浮点传递是合理的；但几何相交判定（球、抛物线）应优先改用整数运算（平方距离、速度平方比较）。浮点判等必须用 ϵ ($10^{-8} \sim 10^{-9}$)。	写 <code>bool onCurve(double a, double b, double x, double y)</code> 用 <code>fabs(...)<1e-8</code> ；写出整数二分的 <code>isqrt(n)</code> ，禁止用 <code>sqrt</code> 直接转整数。
数组、字符串与解析	字符串题需要精确逐字符处理。 <code>stoi</code> 不检查前导零，解析题（IP 地址、表达式格式）必须实现解析器。读一整行用 <code>getline</code> ，注意 <code>cin>></code> 后接 <code>getline</code> 会读到空串。	实现解析器：读入格式串严格拒绝空段、前导零、越界值；对比 <code>stoi</code> 的行为差异。
递归与调用栈	树深度达到 $3 \times 10^5 \sim 5 \times 10^5$ 时，默认栈空间通常不足。正式程序应使用迭代或显式栈；仅在评测环境允许且限制明确时调整栈上限。	写链上 DFS 的递归版和显式栈版，在 $n = 10^5$ 的链上测试递归版是否崩溃。
STL 容器与迭代器	<code>priority_queue</code> （默认大根堆，Dijkstra 需改比较器）、 <code>set/multiset</code> （有序 + 去重/不去重，赛道修建式配对）、 <code>map</code> （值域大替代数组）、 <code>bitset</code> （位运算加速、状压 DP）、 <code>vector</code> 扩容导致迭代器失效。	写 Dijkstra 用最小堆；对 <code>multiset</code> 用 <code>lower_bound</code> 找配对元素；解释 <code>erase(val)</code> vs <code>erase(iter)</code> 的区别。
运算符优先级与位运算	<code>&</code> 、 <code> </code> 和异或运算符的优先级低于 <code>==</code> 、 <code>!=</code> 。if $(x \& 1 == 0)$ 不表示最低位判定； $1 \ll k$ 在 $k \geq 31$ 时可能触发未定义行为， $1ULL \ll 64$ 的移位量也非法。	解释 $1 \ll 63$ (<code>int</code> 左移 UB) vs $1LL \ll 63$ ；写 <code>popcount</code> 用 <code>__builtin_popcountll</code> ；用位运算枚举子集。
C++14 版本与编译选项	以 <code>-std=c++14</code> 为基线。 <code>std::gcd</code> （C++17）、结构化绑定（C++17）不可用。 <code>__int128</code> 是 GCC 扩展，可用于中间计算但不可用于 IO。	用 <code>g++ -std=c++14 -O2 -static</code> 编译代码；确认未使用 C++17 特性。
Linux、计时与调试工具	常见 Linux 命令、文本编辑工具、 <code>g++</code> 选项、 <code>time</code> 和 GDB 可用于定位 CE、RE、TLE、栈溢出、越界及首个错误状态。	能用 <code>pwd/ls/cd/cp/mv</code> 、 <code>time ./sol < in</code> 、 <code>g++ -g -O0</code> 与 GDB 的 <code>run/break/next/print/bt</code> 完成一次崩溃定位；再用正式 <code>-O2 -std=c++14 -static</code> 复测。
STL 与算法模板库	掌握 <code>pair/tuple</code> 、 <code>set/multiset</code> 、 <code>map/multimap</code> 、 <code>deque</code> 、 <code>priority_queue</code> 、 <code>bitset</code> 及迭代器，并掌握 <code>sort/stable_sort/lower_bound/upper_bound/nth_element</code> 。容器操作复杂度与迭代器失效规则需要同时核对。	给每个容器写出插入、查询、删除的典型复杂度；不用 C++17 结构化绑定遍历 <code>map</code> ；构造一个比较器不满足严格弱序的反例。
引用、指针、结构体与对象生命周期	递归邻接表、数组传参、结构体排序和动态分配会涉及引用/指针；悬空引用、未初始化指针和越界访问会直接产生 UB。	写一个接收 <code>const vector<int>&</code> 的函数并解释为何避免复制；指出返回局部变量地址、 <code>vector</code> 扩容后继续使用元素引用这两种错误的生命周期问题。

知识点	适用范围与要点	掌握要求
类 (class) 与运算符重载	结构体成员对齐与布局模拟需要理解 struct 内存模型。运算符重载用于自定义排序比较或 min-plus 矩阵乘法。	写一个 <code>struct Mat{int a[2][2];};</code> 并重载 <code>*</code> 实现 min-plus 矩阵乘法 ($c_{ij} = \min_k(a_{ik} + b_{kj})$), 用于树上倍增合并 DP。

2 基础算法与问题求解

知识点	适用范围与要点	掌握要求
模拟与枚举	模拟常用于第一题及部分分，须按题意逐步执行环形移动、日期推进或递归结构生成。 $n \leq 10 \sim 20$ 时可考虑子集、排列或棋盘状态枚举。剪枝影响实际运行量，但不自动改变最坏复杂度。	把日期模拟从“逐日”改为“按月/按年跳”，说明为何能加速并验证历法切换边界。
贪心	最小编号优先分配、配对贪心（multiset 中最小 + 最小 $\geq$ 阈值）、截止期逆序调度（每次选剩余叶子中截止期最大的）。贪心必须有交换论证或数学证明，不能靠直觉。	对区间最少点覆盖，证明“按右端点排序选右端点”正确，构造“按左端点贪心”的反例。
* 贡献计算	把总答案拆成每个元素、位置、边或区间的独立贡献求和： $ans = \sum_i val_i \times count_i$ 。常见场景：所有子数组的 max/min/sum 之和、每个结点对多少条路径有贡献。配合单调栈求左右边界、前缀和、差分等手段，常能把 $O(n^2)$ 降到 $O(n)$ 。	用单调栈求每个元素作为最大值的左右边界，计算以它为最大值的子数组个数并验证总和；用贡献式推导“所有子数组元素和”中 a_i 被包含的次数。
二分答案	“最大化最小值”或“最小化最大值”的默认框架。判定函数必须满足单调性。典型场景：二分赛道长度、二分完成天数、二分花费。外层 $\log R \approx 30$ 轮，每轮判定可能需要 $O(n)$ 或 $O(n \log n)$ 。	写出整数二分模板（闭区间不变量），解释中值取 $1+(r-l+1)/2$ vs $1+(r-l)/2$ 的场景差异，处理无解和全可行边界。
* 三分查找	目标函数在区间内单峰或单谷，且能比较两个位置的函数值时使用。每轮取 m_1, m_2 比较 $f(m_1)$ 与 $f(m_2)$ ，每次扔掉极值不在的 $1/3$ 区间。整数三分收缩到区间 ≤ 3 后枚举收尾，实数三分按精度或固定次数（如 60 次）停止。	写整数三分在 $[0, 10]$ 上找单峰函数极值；说明为什么不满足单峰性时三分会失败；构造一个反例（如含平台的函数）验证。
前缀和、差分与树上差分	前缀和 $O(1)$ 回答区间和；差分 $O(1)$ 区间加法最后扫描还原。树上差分：点差分在 u, v 处 +1、LCA 处 -1、parent[LCA] 处 -1；边差分在 u, v 处 +1、LCA 处 -2。	在树上实现点差分和边差分的 DFS 还原，构造一条路径验证每个点/边的统计次数。
双指针、滑动窗口、单调队列	时间窗问题、区间最值滑动。三队列思想（初始有序序列 + 切出两段各自单调）可以替代优先队列，把 $\log n$ 降到 $O(1)$ 。每个元素至多入队一次、出队一次是均摊 $O(n)$ 的核心。	证明“每次取出的最大元素长度单调递减”和“切出的两段分别单调递减”，说明为什么三个队列可替代优先队列。
离散化	值域 10^9 但有效值只有 10^5 个时用离散化。sort+unique+lower_bound 完整流程。在线出现的新坐标不能事后补表。	写离线收集所有坐标 $\rightarrow$ 排序去重 $\rightarrow$ lower_bound 映射的完整代码，验证映射后索引范围。
扫描线	把区间/矩形的开始、结束变成按坐标排序的事件，扫描时用计数器、堆、BIT 或线段树维护活动集合。必须规定同坐标事件顺序及闭区间/半开区间语义。	写区间最大重叠数：将 $[l, r]$ 转为 $(l, +1), (r+1, -1)$ ；再写矩形面积并的事件与离散化框架，解释同 x 事件为何应合并后再累计面积。
分治算法	问题能拆成规模近半的独立子问题并线性合并时使用。归并排序、逆序对、CDQ/整体二分的基础都是 $\square$ 递归处理两半 + 只统计跨区间贡献 $\square$ ；递推式 $T(n) = 2T(n/2) + O(n) = O(n \log n)$ 。	实现归并排序并同时统计逆序对；画出递归树，计算递归栈 $O(\log n)$ 与辅助数组 $O(n)$ ，避免只写时间不写空间。

知识点	适用范围与要点	掌握要求
经典排序与稳定性	需要掌握归并、快速、堆、桶、基数排序。比较排序一般为 $O(n \log n)$ ；桶/基数依赖值域或位数。快速排序实现版最坏 $O(n^2)$ ，比赛优先用有最坏保证的 <code>std::sort</code> （内省排序）。	独立写归并排序、堆排序、计数/桶排序与 LSD 基数排序；对每种说明稳定性、额外空间、值域条件，并构造全相等或已有序输入检查实现快排退化。
搜索、剪枝与记忆化	DFS/BFS 是暴力和部分分基础。可行性剪枝、最优性剪枝、搜索顺序优化必须保持正确性；记忆化要求状态足以决定后续。双向 BFS、迭代加深、启发式搜索适用于分支大而答案深度较浅或有可靠估价函数的状态图。	为同一小题写朴素 DFS、带可证明剪枝 DFS 和记忆化搜索，分别给最坏复杂度；实现双向 BFS 与 IDDFS，并解释 A* 何时需要可采纳启发函数才能保证最优。
* 折半搜索 (Meet in the Middle)	$n \leq 30 \sim 40$ 且 2^n 太大但 $2^{n/2}$ 可接受时使用。将元素集均分为两半，分别枚举所有子集并排序，再对一侧的每种状态在另一侧二分/双指针找最优配对。典型场景： $n \leq 40$ 的四景点选择和、子集和等于目标值。	写“ n 个数选若干使和恰好为 K 的方案数”的折半搜索版 ($n \leq 40$)，枚举前 $n/2$ 和后 $n/2$ 各自所有子集和，排序后用双指针合并计数。
倍增	树上跳 k 级祖先 (LCA 预处理)、树路径 min-plus 矩阵倍增合并、DP 矩阵倍增。核心是把 k 按二进制拆位，每次合并两段 2^{j-1} 的结果。	写出 $up[j][v] = up[j-1][up[j-1][v]]$ 转移式；实现 LCA 查询。
复杂度分析	不只看循环嵌套层数，还要分析：每个元素被处理的总次数（均摊）；分层图 DP 状态数 $N \times K$ ；线段树每次 $O(\log n)$ 的总操作数；递归/分治的主定理。	分析分层 DP $O(K(N+M))$ 在 $K=50, N=10^5$ 时的可行性；对比 $O(n \log n + m)$ vs $O(m \log n)$ 的适用场景。
* 部分分与测试点设计	测试点表通常按规模、固定参数、图结构或数值性质分组。部分分方案应分别利用 $n \leq 10 \sim 20$ 、 $m=0$ 、 $K=0$ 、 $k=1$ 、链结构、全正数或小值域等条件，并计算各组覆盖范围。	仅根据数据范围表列出各档测试点可用的算法、时间复杂度、空间复杂度及分值并集。
* 边界数据构造	提高组关键边界：零权边/环、 $k=64$ 移位 UB、同值破平规则、严格 vs 非严格不等式、 $k_i=0$ 或 1 的概率边界。全相等/全负/退化树链/稠密图/最大值边界也需要覆盖。	为廊桥分配式问题构造“所有事件同时到达”的数据；为策略游戏式问题构造“A 区间全正 + B 区间全负”的符号边界。
* 循环不变量与正确性	循环或递归应有明确的不变量。DP 需要区分“前 i 个”与“以 i 结尾”等状态语义；贪心需要交换论证；二分需要固定开闭区间及循环前后的边界含义。	为核心循环注明进入条件与结束后的变量含义；为 DP 状态写出完整定义，并据此推导初始化、转移和答案位置。

3 字符串

知识点	适用范围与要点	掌握要求
字符串解析与格式转化	适用于复杂度格式、IP/端口及类型定义等结构化文本。解析过程需要处理分段数量、前导零、值域和非法字符；stoi 不能代替完整的格式校验。	实现表示 $O(n^{100})$ 与 $O(1)$ 的格式解析，分别得到指数 100 与 0，并拒绝字段缺失或指数格式非法的输入。
* 栈规约（相邻消除）	删除相邻相同字符的最终规约结果唯一。子串可完全消除 $\iff$ 两端前缀规约状态相同。可用精确状态编号（每节点至多 $ \Sigma $ 条子边）替代哈希，规避碰撞风险。	证明“规约唯一性”（无论删除顺序如何，最终结果相同），用所有长度 ≤ 10 的三字符串穷举验证。
Trie（前缀树）	大量字符串的前缀查询、字典维护。字符集大小决定节点空间（ $ \Sigma \times \text{节点数}$ ）。数组式 <code>next[node][c]</code> 比指针式更紧凑，但要注意 $ \Sigma $ （小写 26、扩展可能更大）。	用数组写 Trie，支持插入、精确查找、前缀计数；估算 10^5 个总长 5×10^6 的字符串在 26 元字符集下的内存。
字符串哈希	快速比较子串相等。约定半开区间 $[l, r)$ ： $\text{hash}(l, r) = H[r] - H[l] \times \text{base}^{r-l}$ 。双模降低碰撞概率但不能绝对避免；确定性场景用精确状态编号或双哈希强校验。	推导哈希公式；根据实际比较次数估计生日碰撞风险，不能只看字符串长度下结论；实现双模哈希并说明它仍是概率算法。
KMP / Z-algorithm	KMP 是线性匹配算法；前缀函数 $\pi[i]$ 表示最长相等真前后缀。若 $p = n - \pi[n - 1]$ 且 $n \bmod p = 0$ ，则 p 是最短循环节长度。Z-algorithm 求每个后缀与原串的 LCP，是 S10 字符串重复段分析的常用等价工具。	实现 KMP 求前缀函数和匹配；实现 Z-algorithm；对比两者在重复段检测场景的适用性。
Manacher（马拉车）	在线性时间内求以每个中心为中心的最长奇/偶回文半径，进而统计回文子串、求最长回文或回答离线回文判定。哨兵改造串时必须明确原下标与新下标的映射。	分别实现奇偶半径版或统一插入分隔符版；用空串、单字符、全相同和偶长度回文检查边界，并说明为何总扩展次数为 $O(n)$ 。
* 扩展 KMP / ExKMP	先求模式串的 Z 数组，再在线性时间求文本每个后缀与模式串的 LCP（extend 数组）；适合批量前缀匹配、边界比较和重复结构。	实现 Z 与 extend 两阶段，解释当前最右匹配盒的复用；用模式比文本长、全相同和无匹配数据验证 $O(n + m)$ 时间、 $O(n + m)$ 空间。
* AC 自动机	Trie 上建立失败指针，把多模式匹配转成自动机遍历。定长数组补全转移时构建和空间为 $O(S \Sigma)$ ，匹配 $O(T + \text{occ})$ ；稀疏边可降空间，但转移查询复杂度取决于映射与缓存。	实现 BFS 建 fail、沿 fail 树累计出现次数；说明重复模式、一个模式是另一个后缀、字符集过大时数组转移的空间处理。
* 后缀数组与 LCP	倍增排序构造 SA 常为 $O(n \log n)$ ，Kasai 求 height/LCP 为 $O(n)$ ；适合后缀排序、最长重复子串和结合 RMQ 的后缀 LCP。	实现倍增 SA（计数排序优化可保持 $O(n \log n)$ ）与 height；用全相同、严格递增字符和重复前缀检查排名、下标和哨兵边界。
括号序列与语法 DP	规范化区间 DP 的核心：枚举“第一个原子括号块的右括号”作为唯一分割点，消除同一串的多种语法推导导致的重复计数。将所有中间状态（SAS、A+S 等）写成互斥类。	构造一个会被“枚举任意 AB 分割”重复计数的括号串，说明首原子规范化为何唯一。

4 数据结构

知识点	适用范围与要点	掌握要求
栈、队列、双端队列	BFS (队列)、括号/表达式解析 (栈)、0-1 BFS (deque)、滑动窗口最值 (单调队列)。元素最多进出一次是均摊核心。	实现循环队列 (数组实现)，解释判空判满为什么要留一个空位。
堆与优先队列	Dijkstra 取最小距离、逆序调度取最大截期、多路归并取最大。默认大根堆，最小堆需改比较器。Dijkstra 中同一点可能入堆多次，旧状态必须跳过 (懒删除)。	写 Dijkstra 的懒删除堆；实现逆序调度：维护叶子集合，每轮取截期最大的安排到最后。
并查集	用于连通性判定、逆序加边或加点、带权关系和奇偶约束。二倍点并查集可表示 $x \oplus y = 1$ 等否定关系，也可用于缩点后的分量维护。	实现带路径压缩的 find 和按大小合并的 merge；实现二倍点并查集，并说明变量 x 与 $\neg x$ 属于同一集合时所表示的矛盾关系。
树的孩子兄弟表示法	用 $\square$ 第一个孩子 + 下一个兄弟 $\square$ 把任意有根树表示成二叉树，适合递归结构、语法树和只允许二叉指针字段的存储；它不是原树的左右儿子。	将一棵每个点儿子数不同的有根树转换为孩子兄弟表示，再还原儿子列表；说明空间 $O(n)$ 、遍历仍为 $O(n)$ 。
树状数组 (Fenwick)	单点修改与前缀和查询。BIT 上二分可查找第 k 个有效位置 (维护空位或已删除位)。代码量和常数通常小于线段树；区间修改需结合差分技巧。	实现 add/sum；实现 BIT 二分；用 BIT 模拟序列中”删除第 y 个元素 $\rightarrow$ 后方前移 $\rightarrow$ 末尾追加”的过程。
线段树	区间最值/区间和/区间修改 (懒标记)。静态数组版线段树需开 $4n$ 而非 $2n$ (非 2 的幂时不够)；懒标记的 pushdown 必须在访问子节点前完成。	实现线段树 (支持区间加、区间和查询)；实现懒标记；实现线段树二分 (在线段树上找第一个满足条件的位置)。
ST 表 (稀疏表)	静态 RMQ，预处理 $O(n \log n)$ 、查询 $O(1)$ 、不支持修改。常用于区间 min/max 预处理后 $O(1)$ 回答大量询问。	实现 ST 表，解释 $\log_2$ 预处理和查询时两段重叠覆盖的原理 (重叠不影响 min/max 的幂等性)。
哈希表与冲突处理	会构造整数、字符串、复合键哈希；理解拉链法和开放寻址。unordered_map 查找期望 $O(1)$ 、最坏 $O(n)$ ，需要最坏保证或键值可离散化时改用数组/排序/map。	实现拉链哈希或开放寻址表；给出装载因子、扩容、删除标记的处理；构造大量同桶键并解释为何不能把期望复杂度写成最坏保证。
单调队列 / 单调栈	滑动窗口最值 (窗口大小固定)、DP 优化 (转移窗口是坐标/时间区间)。单调栈求每个元素左右第一个更大/更小的位置，配合贡献计算统计”该元素作为最值的区间数”。三队列 (初始 + 切出两段) 是利用”切出部分单调递减”的隐含单调性把堆换成 $O(1)$ 取最值。	实现单调队列求滑动窗口最大值 (deque 存下标)；用单调栈求每个元素作为最小值的左右边界，验证”所有子数组最小值之和”的贡献公式。
* 分块、块状链表与莫队	分块、块状链表和离线处理适用于需要灵活维护序列或静态区间询问的场景。序列按约 $B \approx \sqrt{n}$ 划块，单点修改、区间查询常做到 $O(1)$ 与 $O(\sqrt{n})$ 的组合；块状链表用块内数组和分裂/重构支持插入；莫队把静态区间询问离线排序，通过四个端点移动维护答案。	写静态莫队并证明端点移动为 $O((n+q)\sqrt{n})$ 量级；写块状链表的定位、插入和超长块重构，给出典型均摊 $O(\sqrt{n})$ ；说明莫队不能直接处理在线询问，带修改版本还需时间维。
* 可持久化线段树	每次修改只复制根到叶路径，旧版本结点不变；主席树用前缀版本差回答静态区间第 k 小。离散化后预处理常为 $O(n \log V)$ 时间和空间，查询 $O(\log V)$ 。	写版本根数组和动态开点，检查结点池上界约 $(n+q)(\log V+1)$ ；解释查询 $[l, r]$ 为什么使用版本 $root[r] - root[l-1]$ 的计数差。

知识点	适用范围与要点	掌握要求
* 左偏树 / 可合并堆	在保持堆序的同时维护零路径长，使两堆合并沿右链递归；合并、插入、删除堆顶为 $O(\log n)$ ，空间 $O(n)$ 。	实现 <code>merge(x,y)</code> ，在交换左右儿子后更新距离；构造大量单点堆连续合并并核对堆序和父子关系。
笛卡尔树	同时满足下标的二叉搜索树性质和值的堆性质；单调栈 $O(n)$ 建树。可把区间最值的递归分治结构显式化。相等值时的优先规则必须固定，否则树形不唯一。	用单调栈建最小笛卡尔树，验证中序遍历还原原下标序列、父值不大于子值；写出 $O(n)$ 时间和 $O(n)$ 空间分析。
平衡树 / 有序集合	<code>std::set/multiset</code> 用于动态有序集合，配对问题常使用 <code>lower_bound</code> 。AVL、Treap、Splay 可用于实现排名、第 k 小或自定义聚合信息；仅需有序集合操作时可使用 STL。	用 <code>multiset</code> 正确完成重复值配对；再写支持插入、删除一个、排名和第 k 小的旋转 Treap，说明期望 $O(\log n)$ 与随机优先级退化风险。

5 图论

知识点	适用范围与要点	掌握要求
图存储与遍历	稀疏图用邻接表/前向星；稠密图 ($V \leq 300$) 用邻接矩阵。BFS 标记在入队时做（非出队时）；DFS 注意递归深度。有向/无向建图不同。	对有向图和无向图分别写 BFS 求最短路和 DFS 判环。
拓扑排序与 DAG DP	有向无环图 (DAG) 的线性排序。DAG 上可 DP 计数路径数或求最长路。典型场景：函数调用依赖分析、排水系统分流、最短路 DAG 上的路径计数。 DAG 反向处理 ：当操作序列中的函数在 DAG 上存在共享子调用时，从序列末尾向前处理，维护“后续乘数累积”以避免指数级展开。	实现 Kahn 算法 (BFS 维护入度) 和 DFS 后序反转两种拓扑排序；在含环图上验证两者都能检测环。写出函数调用问题的反向处理：先拓扑序求每个函数的乘数贡献，再从执行序列末尾向前累加。
单源最短路	Bellman-Ford 为 $O(VE)$ ，可处理负边，并以第 V 轮仍可松弛作为可达负环的判据；堆优化 Dijkstra 为 $O((V+E)\log V)$ ，仅适用于非负边；SPFA 的最坏复杂度仍为 $O(VE)$ 。	分别实现 Bellman-Ford 与 Dijkstra，并以含负边的数据验证 Dijkstra 的适用条件。SPFA 的严格负环判定可记录当前最短路边数，或改用 Bellman-Ford 的第 V 轮松弛；单个顶点的入队次数不能单独构成证明。
Floyd-Warshall 与传递闭包	$O(V^3)$ 时间、 $O(V^2)$ 空间求全源最短路，允许负边但存在负环时最短路无定义；结束后 $d[i][i] < 0$ 表示 i 可达某负环并能回到 i 。布尔 Floyd 可求可达性。	实现 Floyd 并把 k 放最外层；用 INF 防止无穷相加溢出；说明为什么原地更新仍正确。
单源次短路	维护每点最短与严格次短距离，或在最短路 DAG/状态图上扩展 $\square$ 已偏离 $\square$ 状态。必须先确认题目对 $\square$ 次短 $\square$ 是否允许同长度不同路径、重复点/边。	写双距离 Dijkstra，构造多条等长最短路检查 $\square$ 严格大于最短 $\square$ 与 $\square$ 第二条不同路径 $\square$ 两种定义的差异，给出 $O((V+E)\log V)$ 时间和 $O(V+E)$ 空间。
0-1 BFS	边权仅为 0 或 1 时用 deque：0 边推前端，1 边推后端。复杂度 $O(V+E)$ ，比 Dijkstra 少 $\log$ 因子。不能用于边权含 2 或更大的图。	实现 0-1 BFS，并与 Dijkstra 对比；用含权值 2 的数据说明该算法只适用于 0/1 边权。
最小生成树 (Kruskal / Prim)	Kruskal (并查集 + 排序边, $O(E\log E)$) 适合边表；堆优化 Prim 为 $O(E\log V)$ ，邻接矩阵 Prim 为 $O(V^2)$ ，适合稠密小图。图不连通时得到的是最小生成森林，不存在 MST。	实现 Kruskal 与两种 Prim；检查最终选边数是否为 $V-1$ ；解释负边不影响 MST 正确性。
欧拉道路与欧拉回路	所有边恰走一次。无向图看奇度点数与非零度点连通性；有向图看入度/出度差与弱连通性。Hierholzer 算法 $O(V+E)$ ，必须按 边编号 标记无向边的两条弧。	写有向、无向两版判定与构造；处理孤立点、重边、自环，最后检查输出边数恰为 E 。
强连通分量 (SCC / Tarjan)	有向图的 SCC 缩点后得到 DAG。用途：零环检测、依赖环压缩和分量级 DP；2-SAT 作为更高阶应用单独判断。	实现 Tarjan (DFN+LOW+栈)，解释“DFN = LOW 意味着是 SCC 根”；在零边子图上应用 SCC 检测零环。
桥、割点与双连通分量	无向图树边 (u, v) 是桥当 $low[v] > dfn[u]$ ；非根 u 是割点当存在儿子 v 使 $low[v] \geq dfn[u]$ ，DFS 根需至少两个 DFS 儿子。重边图必须按 边编号 跳反向边，不能按父结点跳过全部平行边。	实现桥、割点与边双连通分量缩点；用重边及 DFS 根只有一个儿子的数据验证父边和根结点处理；连通图缩点后应得到桥树。
二分图判定	染色法 BFS/DFS：相邻点必须不同色。含奇环的图不是二分图。	实现染色法判定二分图；说明为什么含奇环的图不是二分图。

知识点	适用范围与要点	掌握要求
* 二分图最大匹配 / 匈牙利算法	匈牙利增广路算法逐个处理左部点, 每次 DFS 寻找增广路, 时间 $O(VE)$ 、空间 $O(V + E)$; Hopcroft-Karp 为 $O(E\sqrt{V})$ 。根据柯尼希定理, 二分图最小点覆盖数等于最大匹配数。	实现 DFS 增广, 访问标记必须每轮更新; 处理非连通图、左右部大小不同与重边; 用 $\square$ 最大匹配数 = 最小点覆盖数 $\square$ 构造覆盖集合。
* 2-SAT	每个布尔变量拆成两个文字, 限制转成蕴含边; 可满足当且仅当任一变量与其否定不在同一 SCC, 时间和空间均 $O(V + E)$ 。	把 $\square$ 至少一个 $\square\square$ 至多一个 $\square\square$ 二者相同/不同 $\square$ 分别转成边; Tarjan/Kosaraju 后按 SCC 拓扑关系构造一组赋值并验证原约束。
* 最大流 / 最小割	Dinic 用 BFS 分层图和 DFS 当前弧增广; 一般图常用上界 $O(V^2E)$, 空间 $O(V + E)$ 。最大流等于最小割, 建模时容量、反向边和无穷容量上界必须可靠。	实现 Dinic (每条正向边配反向边), 从残量网络找最小割两侧; 用 64 位容量, 构造平行边、零容量边和无法到汇点的情况。
* 最小费用最大流	在残量网络中反复寻找单位费用最短的增广路。SPFA 版每次最短路最坏 $O(VE)$; 势能 + Dijkstra 在约化费用非负后每次 $O(E \log V)$ 。若总增广流量为 F , 按单位流上界粗估分别为 $O(FVE)$ 与 $O(FE \log V)$, 所以它不是与数值大小无关的强多项式保证。	实现正反边容量与费用互为相反数的建边; 区分 $\square$ 最大流前提下最小费用 $\square$ 和 $\square$ 只送指定流量 $\square$; 检查负费用边、不可继续增广、费用乘流量溢出及初始势能条件。
差分约束 (Bellman-Ford 应用)	将 $x_v \leq x_u + w$ 表示为边 $u \rightarrow v$ 、权 w , 从而把上界约束转为最短路不等式。加入超级源后, 可用 Bellman-Ford 或 SPFA 求可行解并检测负环; 不小于关系应先统一变形, 再确定边的方向。	将 $x_a - x_b \leq c$ 、 $x_a - x_b \geq c$ 、 $x_a = x_b$ 分别转边; 写 Bellman-Ford $O(VE)$ 的严格可行性判定, 并说明 SPFA 最坏仍为 $O(VE)$ 。
* 平面图对偶	平面图上: 原图多终端最小割 $\iff$ 对偶图边界间隙的非交叉完美匹配最短路。两终端时退化为一次对偶最短路。属 NOI 级延伸。	在 2×2 网格上画出原图边、对偶边、两个异色边界终端和对应的割路径。

6 树论

知识点	适用范围与要点	掌握要求
* 树的直径	两次 DFS/BFS (从任意点到最远点, 再从该点到最远点): 仅适用于非负权边。树形 DP 维护每个子树的最长路径可处理任意权。	实现两次 BFS 求直径, 验证正确性依赖非负权; 构造带负权边的反例。
* 树的重心	删去重心后所有子树大小 $\leq \lfloor n/2 \rfloor$ 。快速求删边后两侧子树的重心: 沿重儿子链向下跳跃 (倍增) $O(\log n)$ 定位。树可能有 1 或 2 个重心, 两个时必相邻。	实现找重心 (DFS 求子树大小, 沿重儿子链下跳); 处理“补树”的重心 (换根后子树大小变化)。
LCA (最近公共祖先)	倍增法预处理 $O(n \log n)$ 、查询 $O(\log n)$ 。用途: 路径拆分 (上行/下行段分别差分)、树上距离查询、路径 DP 矩阵倍增合并。Tarjan 离线 LCA 用于批量查询。	实现倍增 LCA (up[k][v] 表 + 深度数组); 实现树上两点距离查询; 对比 Tarjan 离线 LCA 的适用场景。
DFS 序与子树操作	子树在 DFS 序中是一个连续区间 $[in[u], out[u]]$, 将子树操作转化为区间操作。结合线段树/BIT 维护子树和/子树加。	写 DFS 序预处理, 实现子树加法 + 子树求和。
* 树链剖分	按重儿子把树拆成 $O(\log n)$ 条路径段, 把路径修改/查询转成若干 DFS 序区间; 配合线段树通常每次 $O(\log^2 n)$, 空间 $O(n)$ 。	实现两遍 DFS 的 size/heavy/top/dfn, 处理路径时每次跳较深链顶; 分别验证点权、边权映射以及 LCA 位置是否应计入。
树上差分	见第 2 节“前缀和、差分与树上差分”。提高组核心应用: 将玩家路径拆为上行段和下行段, 在 DFS 过程中用全局桶 (进入前记录、退出后求差) 统计经过每个结点的路径数。	在树上模拟“多条路径从不同起点到不同终点”的场景, 用树上差分 + 桶统计每个结点被经过的次数。
树形 DP	状态为 $f[u][0/1]$ (选/不选 u 时子树的最优值) 最为常见。强制条件的加入方式: 将禁止状态的 DP 初值设为 INF。 u 不选 $\rightarrow$ 儿子必选; u 选 $\rightarrow$ 儿子可选。	写树上最小点覆盖 DP; 加入强制条件 (某点必须选/不选); 冲突判断 (相邻两点都不选 $\rightarrow$ 无解返回 -1)。
* 换根 DP (二次扫描)	第一遍 DFS 求子树信息, 第二遍 DFS 从根向下传递父方向的补树信息。用途: 求每个点到其他所有点的距离和、删边后补树的重心/直径。	写换根 DP 求每个点到其他所有点的距离和。
* 基环树	连通无向图 $m = n$ (恰好一个环)。处理技巧: 拓扑删叶找环 $\rightarrow$ 枚举删环上每条边退化为树 $\rightarrow$ 每次做树 DFS 取最优。或 DFS 过程中做一次回溯决策。	拓扑删叶 (反复删度为 1 的点) 找环; 枚举删环上每条边做树 DFS。
树上倍增与矩阵合并	树路径上的转移写成 2×2 或 3×3 矩阵 (min-plus 半环乘法), 倍增合并路径区间。用于“距离不超过 k 的树上最小代价传输”类问题; 更一般的动态 DP 不在本表主范围。	写出 $k = 2$ 时的转移矩阵; 解释 min-plus 矩阵乘法的结合律; 实现树上倍增合并矩阵。

7 动态规划

知识点	适用范围与要点	掌握要求
状态定义与初始化	DP 状态必须明确表示 $\square$ 前 i 个 $\square$ 或 $\square$ 以 i 结尾 $\square$ 等范围。求最大值时, 不可达状态设为 $-\infty$; 求最小值时设为 $+\infty$ 。 $\square$ 恰好 $\square$ 与 $\square$ 至多 $\square$ 的初始化不同。	分别写出 $\square$ 恰好 j 次申请 $\square$ 和 $\square$ 至多 j 次申请 $\square$ 的初始化; 用全负网格取数验证不可达状态不能初始化为 0。
背包 DP	0/1 背包 (倒序)、完全背包 (正序)、多重背包 (二进制拆分或单调队列优化)。用途: 判定面额冗余 (等价货币系统)、计数型背包 (选木棍拼多边形)。	写 0/1 和完全背包, 解释循环方向不同的原因; 用单调队列实现多重背包 $O(nC)$ 。
区间 DP	枚举长度 $\rightarrow$ 枚举左端点 $\rightarrow$ 枚举分割点, $n \leq 500$ 时 $O(n^3) \approx 1.25 \times 10^8$ 。规范化技巧: 用“第一个原子块的结束位置”作为唯一分割点, 消除同一结果的多条语法推导。	写括号序列的规范化区间 DP, 设计互斥的状态类别, 构造一个会被“任意 AB 分割”重复计数的串。
状态压缩 DP	$n \leq 18 \sim 20$ 是信号。预处理每对/每组元素确定的覆盖 $mask$, $dp[mask]$ 从第一个未覆盖位转移 (确保每条覆盖都被且仅被考虑一次)。枚举子集用 $sub = (sub-1) \& mask$ 。	写状压 DP 模板; 写枚举子集; 在 $n \leq 18$ 的覆盖问题上对比“枚举 $mask$ 中每个元素的覆盖”和“只从最低位转移”的效率差异。
树形 DP	见第 6 节树论。提高组的树形 DP 常结合: 强制条件 INF 设定、子树信息与补树信息合并、路径 min-plus 矩阵倍增、子树内配对。	完成“每次询问强制两个结点的状态, 求树的最小点覆盖”的 $O(n)$ 暴力 DP 实现。
* 概率 / 期望 DP	S10 的 NOIP 2016《换教室》直接涉及基本概率 DP, 故保留为真题边界。核心是状态含义、全概率分解与期望线性性。	推导完整转移公式 (含四种位置组合的概率乘积), 与概率为 0/1 的确定性版本对比; 说明该条来自真题。
分层 DP / 多维 DP	“不超过最短路 $+K$ ” $\rightarrow$ 按额外代价 $0..K$ 分层, 每层在 DAG/图上 DP。转移时额外代价增量由最短路保证 ≥ 0 。“二进制序列的权值和” $\rightarrow$ 按位从低到高 DP, 状态记录已放位置数、当前进位、已定 1 的个数。	写出分层 DP 的转移: $u \rightarrow v$ 时额外代价增加 $\Delta = dist[u] + w - dist[v]$ (最短路保证 $\Delta \geq 0$); 零环导致无穷多方案 $\rightarrow$ 需 SCC 检测。
DP 常用优化	包括滚动数组、前缀最值/前缀和、单调队列、数据结构优化、值域驱动的伪多项式 DP、交换枚举次序消去一维, 以及把形如 $dp_i = \min_j (k_j x_i + b_j)$ 的转移改写成直线最值查询。斜率与查询点均单调时可用单调凸包做到总 $O(n)$; 否则需二分凸包或李超树等结构, 不能仍声称 $O(n)$ 。	把一个 $O(nW^2)$ 窗口转移优化为单调队列 $O(nW)$; 推导一个斜率优化转移, 明确直线斜率、截距、查询横坐标、取 min/max 及相等斜率处理; 分别计算优化前后时间与空间。
* 进位 / 逐位组合 DP	按二进制位从低到高处理: 枚举当前位使用次数 $t \rightarrow$ 当前位总数 = 进位 $+t \rightarrow$ 贡献 popcount 的 1 $\rightarrow$ 新进位 = 总数/2。每步乘组合数 $C(n-j, t)$ 和权值的 t 次幂。	推演小规模进位 DP 的全过程并与枚举结果对照; 解释处理完最高位后仍需计入最终进位 popcount 的原因。

8 数学

知识点	适用范围与要点	掌握要求
基本运算与边界	max/min、绝对值、向上取整。abs(INT_MIN) 返回值无法由 int 表示 (UB)。快速幂注意指数为 0 或底数为 0。乘法前检查溢出: $\text{limit}/a < b$ 而非 $a*b > \text{limit}$ 。	先转到足够宽的有符号类型再取绝对值; 仅在 $a \geq 0, b > 0$ 且加法不溢出时用 $(a+b-1)/b$, 一般写商余数版向上取整。
位运算	格雷码公式 $g = k \oplus (k \gg 1)$; 位集合分析 $2^{k- S } - n$; $1ULL \ll k$ 在 $k = 64$ 时是 UB (移位量等于类型宽度) 必须特判。popcount 用 <code>__builtin_popcountll</code> 。	写 popcount、lowbit; 用位运算枚举子集; 写出 $k = 64$ 时安全计算 2^{64} 的方法。
质数筛法 (埃氏筛 / 线性筛)	批量筛质数。埃氏筛 $O(n \log \log n)$ 简单快速; 线性筛 $O(n)$ 同时维护最小质因子用于 $O(\log x)$ 质因数分解。含特定数码因数的倍数筛法 (调和级筛) 用于预处理后继合法数。	实现线性筛 (维护 prime[] 和 lp[x]), 解释 <code>i%prime[j]==0</code> 时 break 的原因。
因数分解与约数个数	若 $n = \prod p_i^{a_i}$, 正约数个数 $\tau(n) = \prod (a_i + 1)$, 约数和 $\sigma(n) = \prod (1 + p_i + \dots + p_i^{a_i})$ 。试除分解/枚举约数为 $O(\sqrt{n})$; 批量维护最小质因子后单次分解为 $O(\log n)$ 。约数对在完全平方数处只能计一次。	写枚举全部约数并排序、由质因数分解递归生成约数、求 $\tau(n)$ 和 $\sigma(n)$; 用 $n = 1$ 、完全平方数和大质数验收。详细数量级见 CSP-S-因子与约数个数.md。
* 整除分块 / 数论分块 (阶段补充)	对 $q = \lfloor n/l \rfloor$, 所有 $i \in [l, \lfloor n/q \rfloor]$ 的 $\lfloor n/i \rfloor$ 相同, 因此可按商值段跳跃; 不同商只有 $O(\sqrt{n})$ 个。常用于求 $\sum_i \lfloor n/i \rfloor$ 、整除对计数和带前缀和的分段求和。	写循环 <code>for(l=1;l<=n;l=r+1) r=n/(n/l)</code> , 证明段数 $O(\sqrt{n})$ 、额外空间 $O(1)$; 两个或多个 floor 同时出现时, 右端点取各自不变区间右端点的最小值。
gcd / exgcd	约分、分数化简。扩展欧几里得 exgcd(a,b,x,y) 求 $ax + by = \text{gcd}(a,b)$ 的一组解, 进而求模逆元 ($\text{gcd}(a,M) = 1$ 时) 或解线性同余方程。	实现 exgcd; 用其求 a 在模 M 下的逆元; 计算 $15x \equiv 1 \pmod{26}$ 。
模运算与快速幂	答案对 $10^9 + 7$ 或 998244353 取模。快速幂 $O(\log k)$ 。减法先 +MOD、乘法用 <code>1LL*a*b%MOD</code> 。组合数取模: 若模数为质数用阶乘 + 逆元 $O(1)$ 查询; 否则用杨辉三角递推取模。	写快速幂迭代版; 在 p 为质数且预处理范围 $n < p$ 时写 $C(n,k) \bmod p$ 的 $O(1)$ 查询 (阶乘 + 逆元), 并解释 $n \geq p$ 时为何不能直接照搬。
同余、模逆元与裴蜀定理	模逆元三种求法: 费马小定理 (M 为质数且数不被 M 整除)、exgcd (M 任意但需互质)、质数模下递推求 $1..n < M$ 的逆元。裴蜀定理: $ax + by = \text{gcd}(a,b)$ 有整数解。互质正整数的最大不可表示数为 $ab - a - b$ 。	证明费马小定理求逆元的条件; 写出递推求 $1..n$ 逆元的代码; 计算 $a = 7, b = 11$ 的最大不可表示数。
欧拉函数 (Euler φ)、欧拉/费马/威尔逊定理	$\varphi(n) = n \prod_{p n} (1 - 1/p)$; $\text{gcd}(a,n) = 1$ 时 $a^{\varphi(n)} \equiv 1 \pmod{n}$ 。费马小定理是质数模特例。威尔逊定理 $(p-1)! \equiv -1 \pmod{p}$ 当且仅当 p 为质数, 但通常不是高效判素算法。	线性筛求 $1..N$ 的 φ ; 逐条写出定理前提并构造 $\square$ 不互质 $\square \square$ 模数合数 $\square$ 的反例, 禁止无条件把指数按 $\varphi(M)$ 取模。
线性同余与中国剩余定理	$ax \equiv b \pmod{m}$ 有解当且仅当 $\text{gcd}(a,m) \mid b$; 解模 $m/\text{gcd}(a,m)$ 唯一。互质模数用经典 CRT; 模数不互质时逐式合并并先检查余数相容, 合并的 lcm 乘法要防溢出。	用 exgcd 解线性同余; 实现互质 CRT 和非互质扩展合并, 构造 $\square$ 无解 $\square \square$ 模数不互质但有解 $\square \square$ lcm 超 64 位 $\square$ 三组边界。
* Lucas 定理 (阶段补充)	它是质数模组合数的常见后续: 对质数 p , $C(n,m) \equiv C(n \bmod p, m \bmod p) C(\lfloor n/p \rfloor, \lfloor m/p \rfloor) \pmod{p}$ 。预处理 $0..p-1$ 阶乘后, 单次 $O(\log_p n)$ 、空间 $O(p)$; 只适合 p 可预处理。	实现 Lucas 递归/循环, 处理某一位 $m_i > n_i$ 时答案为 0; 说明模数合数不能直接使用, 扩展 Lucas 属另一套质因数分解与 CRT 算法。
* 原根与 BSGS	理解模质数乘法群、元素阶和原根; BSGS 用 meet-in-the-middle 解离散对数 $a^x \equiv b \pmod{m}$, 经典互质版本期望 $O(\sqrt{m})$ 时间和空间 (哈希表)。	实现互质条件下的 BSGS 并处理无解情况; 说明哈希复杂度是期望值, $\text{gcd}(a,m) \neq 1$ 时需使用扩展 BSGS。
多重集合、等价关系与基础计数	会区分集合/多重集合; 多重集排列数为 $n! / \prod c_i!$ 。等价关系必须满足自反、对称、传递, 等价类构成划分; 并查集正是维护等价类的算法实现。	推导多重集排列/组合的小例子; 判断给定关系是否为等价关系并列出现等价类; 解释为何只满足对称性不能直接用并查集合并。

知识点	适用范围与要点	掌握要求
错排列、圆排列与二项式定理	错排列 $D_n = (n-1)(D_{n-1} + D_{n-2})$ ，也可由容斥推导；互不相同元素的圆排列为 $(n-1)!$ （若镜像也视为相同需另除 2，且要处理小 n ）。二项式定理连接组合数、卷积和逐项贡献。	推导错排列递推与 $\sum_{k=0}^n (-1)^k n! / k!$ ；说明圆排列的 $\square$ 旋转同一 $\square$ 条件；用二项式定理求系数并核对杨辉三角。
容斥原理与卡特兰数	容斥必须明确事件交集；只有事件确实互斥时才能只减一次。卡特兰数可计合法括号、栈序列等，但要先建立双射，不能见到括号/树就套公式。S10 中还出现关键边子集容斥与差值 DP。	推导 n 对括号的卡特兰数；写三集合完整容斥；为关键边子集容斥列出每个子集的符号和剩余约束。
高斯消元与矩阵运算	需要掌握向量/矩阵概念、向量运算、矩阵加减乘转置、单位/三角/对称矩阵与高斯消元。高斯消元 $O(n^3)$ 解方程组；实数域选绝对值最大主元，模意义下只对可逆主元求逆。	实现实数与质数模两版高斯消元，区分唯一解/无解/多解；实现普通、min-plus 矩阵乘法和矩阵快速幂，检查维数与单位元。
* 几何与浮点	二维几何涉及点线面位置、一般图形面积与凸包。二维向量叉积用于方向/共线/线段相交，鞋带公式 $O(n)$ 求多边形有向面积，单调链凸包为排序 $O(n \log n)$ 、扫描 $O(n)$ 、空间 $O(n)$ 。整数坐标优先用 long long/ __int128 叉积，浮点才统一 eps。	实现方向判定、含端点/共线重叠的线段相交、鞋带公式和单调链凸包；明确是否保留边界共线点。另处理抛物线参数、球相交平方距离与超速判定，避免无必要的 sqrt。
* 差分不变量与数学转化	操作等价于交换相邻差分 $\rightarrow$ 所有可达序列对应差分多重集合的排列。方差整数公式 $n \sum a_i^2 - (\sum a_i)^2$ 。差分排序后最优排列必为单谷（交换论证：大间隔放两端），转化为“逐差分插左/右”的 DP。	代数证明一次操作只交换相邻差分；写出方差整数公式；用交换论证证明单谷性质的最优性。
整数开方与判别式	求 $\lfloor \sqrt{n} \rfloor$ 用整数二分（mid $\leq n/\text{mid}$ 防溢出）。一元二次方程 $\Delta = b^2 - 4ac$ ， Δ 为非负完全平方数时才有整数解。规范化根式输出：提取 $\sqrt{\Delta}$ 的平方因子，约分有理数部分。	写整数二分开方；写一元二次方程规范化输出（含 $\sqrt{s}$ 的根式形式， $\gcd(p, q, r) = 1$ ）。

9 算法复杂度索引

算法	时间复杂度	空间复杂度	限制与易错点
线性扫描 / 模拟	$O(n)$	$O(1)$ 或 $O(n)$	提高组 n 上限 $3 \times 10^5 \sim 10^6$, 注意 IO 瓶颈。
排序	$O(n \log n)$	$O(\log n)$ 栈	比较器须满足严格弱序; $n > 10^6$ 考虑桶/基数。
归并 / 桶 / 基数排序	$O(n \log n)$ / $O(n + A)$ / $O(d(n + B))$	$O(n)$ / $O(A)$ / $O(n + B)$	桶依赖值域 A , 基数依赖位数 d 与基数 B 。
分治 / 扫描线	常见 $O(n \log n)$	常见 $O(n)$	扫描线的具体复杂度还要乘维护结构操作; 同坐标事件顺序要固定。
二分答案 + 判定	$O(\log R \times T(n))$	取决于判定	判定必须单调; $\log R \approx 30 \sim 60$ 轮。
前缀和 / 差分	预处理 $O(n)$, 查询 $O(1)$	$O(n)$	二维前缀和注意边界减两遍加回一遍。
双指针 / 滑窗	$O(n)$ 均摊	$O(1)$ 或 $O(n)$	每个元素最多进出一次。
单调队列 / 单调栈	$O(n)$ 均摊	$O(n)$	窗口单调性必须成立; 每个元素至多进出一次。
* 分块/块状链表/静态莫队	常见 $O(\sqrt{n})$ 每次 / $O((n + q)\sqrt{n})$	$O(n + q)$	块长需重构; 莫队离线且端点增删要快。
BFS / DFS	$O(V + E)$	$O(V + E)$	标记在入队/入栈时做; $V = 3 \times 10^5$ 注意递归栈。
双向 BFS / IDDFS	约 $O(b^{d/2})$ / $O(b^d)$	约 $O(b^{d/2})$ / $O(d)$	只是典型树状估算; 状态重复、分支数和最短性条件会改变实际量。
Bellman-Ford	$O(VE)$	$O(V)$ 或 $O(V + E)$	负边与负环; 第 V 轮仍松弛才是严格负环信号。
Dijkstra (堆)	$O((V + E) \log V)$	$O(V + E)$	非负权; 懒删除跳过旧堆项。
SPFA	最坏 $O(VE)$	$O(V + E)$	可处理负边, 但不存在稳定的平均复杂度保证。
Floyd	$O(V^3)$	$O(V^2)$	$V \leq 300$ 可行; k 循环必须最外层。
Kruskal	$O(E \log E)$	$O(V + E)$	并查集判环; 排序是瓶颈。
Prim (堆/矩阵)	$O(E \log V)$ / $O(V^2)$	$O(V + E)$ / $O(V^2)$	前者稀疏图, 后者稠密小图。
次短路 (双距离)	$O((V + E) \log V)$	$O(V + E)$	先确认严格次短还是第二条不同路径。
欧拉路 (Hierholzer)	$O(V + E)$	$O(V + E)$	每条边恰使用一次; 无向边按编号配对。
* 匈牙利 / Hopcroft-Karp	$O(VE)$ / $O(E\sqrt{V})$	$O(V + E)$	二分图最大匹配; 不是带权 KM。
* 2-SAT	$O(V + E)$	$O(V + E)$	文字建蕴含图后求 SCC。
* Dinic 最大流	一般上界 $O(V^2 E)$	$O(V + E)$	实际依网络结构; 容量与反向边必须正确。
* 最小费用流 (逐次增广)	势能版常写 $O(FE \log V)$; SPFA 版最坏 $O(FVE)$	$O(V + E)$	F 为总增广流量的整数上界; 费用反向边、负边与溢出。
差分约束	Bellman-Ford $O(VE)$; SPFA 最坏 $O(VE)$	$O(V + E)$	统一成 $x_v \leq x_u + w$; 负环表示无可行解。
Tarjan (SCC/桥/割点)	$O(V + E)$	$O(V + E)$	根割点特判; 重边不能只按父结点跳过。
拓扑排序	$O(V + E)$	$O(V + E)$	仅 DAG; 入度 0 入队。
并查集	均摊 $O(\alpha(n))$	$O(n)$	路径压缩 + 按秩合并; 可维护额外信息。
树状数组 (BIT)	$O(\log n)$	$O(n)$	代码量和常数通常小于线段树; 支持 BIT 上二分。
线段树	$O(\log n)$	$O(4n)$	开 $4n$ 非 $2n$; 懒标记的 pushdown。
* 可持久化线段树	建树/修改/查询 每次 $O(\log V)$	总计 $O((n + q) \log V)$ 结点	旧版本不可修改; 先估结点池。

算法	时间复杂度	空间复杂度	限制与易错点
ST 表	预处理 $O(n \log n)$, 查询 $O(1)$	$O(n \log n)$	静态 RMQ; 不支持修改。
哈希表	期望 $O(1)$, 最坏 $O(n)$ 每次	$O(n + B)$	装载因子、冲突和恶意键会导致退化。
笛卡尔树	建树 $O(n)$	$O(n)$	单调栈; 相等值破平规则要固定。
平衡树	每次期望/保证 $O(\log n)$	$O(n)$	Treap 是期望; AVL/Splay 的保证与均摊口径不同。
* 左偏树	合并/插入/删顶 $O(\log n)$	$O(n)$	维护堆序与零路径长。
离散化	$O(M \log M)$	$O(M)$	M 是收集到的坐标数。
LCA (倍增)	预处理 $O(n \log n)$, 查询 $O(\log n)$	$O(n \log n)$	先跳到同深度, 再一起跳。
* 树链剖分 + 线段树	预处理 $O(n)$, 每次 $O(\log^2 n)$	$O(n)$	点权/边权映射与 LCA 是否计入。
树上差分	$O(n)$ DFS 还原	$O(n)$	点差分和边差分公式不同。
Trie (数组)	建树 $O(S)$, 查询 $O(s)$	$O(S \Sigma)$	字符集大小决定空间。
区间哈希	预处理 $O(L)$, 查询 $O(1)$	$O(L)$	双模降碰撞率; 非绝对证明。
KMP / Z-algorithm	$O(n + m)$ / $O(n)$	$O(m)$ / $O(n)$	匹配过程文本串指针不回退。
Manacher	$O(n)$	$O(n)$	奇偶回文和改造串下标映射不能混用。
* ExKMP / AC 自动机	$O(n + m)$ / $O(S \Sigma + T + occ)$	$O(n + m)$ / $O(S \Sigma)$	AC 稀疏转移可降低字符集空间。
* 后缀数组 + LCP	$O(n \log n)$	$O(n)$	倍增排序与 Kasai; 排名/下标边界。
埃氏筛 / 线性筛	$O(n \log \log n)$ / $O(n)$	$O(n)$	线性筛同时维护最小质因子。
gcd / exgcd	$O(\log \min(a, b))$	$O(1)$	C++14 下需自行实现。
试除分解 / 枚举约数	$O(\sqrt{n})$	$O(1)$ 或 $O(\tau(n))$	完全平方根只计一次; $n = 1$ 特判。
* 整除分块	$O(\sqrt{n})$	$O(1)$	$r = n/(n/l)$; 多个 floor 取右端点最小值。
快速幂	$O(\log k)$	$O(1)$	底数和指数都可能大; 注意取模。
组合数 (预处理)	$O(n^2)$ 递推或 $O(n)$ 阶乘 + 逆元	$O(n^2)$ 或 $O(n)$	阶乘逆元法通常还要求质数模且 $n < p$ 。
* Lucas (质数模)	预处理 $O(p)$, 查询 $O(\log_p n)$	$O(p)$	p 必须为质数且小到能预处理。
* BSGS	期望 $O(\sqrt{m})$	$O(\sqrt{m})$	经典版要求互质; 哈希存在退化风险。
* 单调链凸包	$O(n \log n)$	$O(n)$	叉积防溢出; 共线边界点保留规则必须统一。
斜率优化 DP	单调条件下总 $O(n)$, 一般实现常为 $O(n \log n)$	$O(n)$	先推成直线查询; 相等斜率、min/max 与查询顺序决定实现。
区间 DP	$O(n^3)$ 到 $O(n^2)$	$O(n^2)$	$n \leq 500$ 时 $O(n^3) \approx 1.25 \times 10^8$ 。
状压 DP	$O(2^{2n})$ 或 $O(n3^n)$	$O(2^n)$	$n \leq 18$ 时 $2^{18} = 262144$ 。
树形 DP	固定状态常为 $O(n)$	常为 $O(n)$	子树背包合并可能到 $O(n^2)$ 或更高, 必须按状态与合并循环重算。
背包 DP	$O(nC)$	$O(C)$ 滚动	0/1 倒序、完全正序。
分层图 DP	$O(K(V + E))$	$O(KV)$	$K \leq 50 \sim 100$ 是关键信号。
高斯消元	$O(n^3)$	$O(n^2)$	模意义下改除为乘逆元。

算法	时间复杂度	空间复杂度	限制与易错点
矩阵快速幂	$O(s^3 \log n)$	$O(s^2)$	min-plus 半环用于 DDP。

10 空间复杂度与内存估算

内存按二进制单位估算：1 MiB = 2^{20} 字节，1 GiB = 2^{30} 字节。理论元素数为 $\lfloor M/\text{sizeof}(T) \rfloor$ ；主数组占用宜控制在内存限制的约 75% 以内，其余空间留给程序、其他数组、容器与分配器、递归栈和运行库。下表 □ 建议 □ 均为四舍五入后的保守数量级，不代表评测环境的实际可用上限。

限制	总字节	char (1B)	int (4B)	long long/double (8B)	__int128 (16B)
128 MiB	134,217,728	理论 1.34×10^8 ； 建议 1.0×10^8	理论 3.36×10^7 ； 建议 2.5×10^7	理论 1.68×10^7 ；建 议 1.25×10^7	建议 6.2×10^6
256 MiB	268,435,456	理论 2.68×10^8 ； 建议 2.0×10^8	理论 6.71×10^7 ； 建议 5.0×10^7	理论 3.36×10^7 ；建 议 2.5×10^7	建议 1.25×10^7
512 MiB	536,870,912	理论 5.37×10^8 ； 建议 4.0×10^8	理论 1.34×10^8 ； 建议 1.0×10^8	理论 6.71×10^7 ；建 议 5.0×10^7	建议 2.5×10^7
1 GiB	1,073,741,824	理论 1.07×10^9 ； 建议 8.0×10^8	理论 2.68×10^8 ； 建议 2.0×10^8	理论 1.34×10^8 ；建 议 1.0×10^8	建议 5.0×10^7

- $O(n)$ 只描述增长率，不表示内存一定小：线段树若有两组 `long long tree[4*n]`，主数组就是约 $64n$ 字节；ST 表是 $\text{sizeof}(T) \cdot n(\lfloor \log_2 n \rfloor + 1)$ 字节。
- 二维方阵要先算 $n^2 \text{sizeof}(T)$ 。按 75% 建议值，int 方阵在 128/256/512 MiB/1 GiB 下的 n 约不超过 5000/7000/10000/14000；long long 约不超过 3500/5000/7000/10000。
- `vector<vector<T>>` 还有每行对象和多次分配开销；`map/set/unordered_map` 每元素远大于键值本身。图要按有向弧数算：无向 m 条边通常存 $2m$ 条弧。
- 递归栈既可能计入总内存，又常受独立栈上限约束；全局/BSS、堆和栈都不能假设 □ 免费 □。完整估算、状压和邻接表换算见 CSP-S-空间复杂度与内存估算.md。

11 时间复杂度估算基准

本节只换算增长函数的理论规模；上一节的算法索引用于查询具体算法的时间和空间复杂度，两者不重复列举。估算基准为 **1 秒执行 $5 \times 10^8 \sim 10^9$ 次极简单整数运算**（-02）。稳定线取 2.5×10^8 次，上限线取 9×10^8 次。稳定线以下通常有实现余量；稳定线与上限线之间需要控制常数；超过上限线时应更换算法。随机访存、STL 容器、哈希、递归和多组数据不能按一次简单运算折算。

增长级别	稳定线 ($\leq 2.5 \times 10^8$)	上限线 ($\leq 9 \times 10^8$)	说明
$n!$	$n \leq 11$	$n \leq 12$	$11! \approx 4.0 \times 10^7$, $12! \approx 4.8 \times 10^8$ ；若每排列还做 $O(n)$ 检查，需把乘积重算。
2^n	$n \leq 27$	$n \leq 29$	$2^{27} \approx 1.34 \times 10^8$, $2^{29} \approx 5.37 \times 10^8$ ；只适用于每状态常数工作。
$n2^n$	$n \leq 23$	$n \leq 25$	$23 \cdot 2^{23} \approx 1.93 \times 10^8$, $25 \cdot 2^{25} \approx 8.39 \times 10^8$ ；空间限制往往先于时间限制。
$n^2 2^n$	$n \leq 19$	$n \leq 20$	$19^2 2^{19} \approx 1.89 \times 10^8$, $20^2 2^{20} \approx 4.19 \times 10^8$, $n = 21$ 已超过上限线。
3^n	$n \leq 17$	$n \leq 18$	$3^{17} \approx 1.29 \times 10^8$, $3^{18} \approx 3.87 \times 10^8$ 。
n^5	$n \leq 47$	$n \leq 61$	$47^5 \approx 2.29 \times 10^8$, $61^5 \approx 8.45 \times 10^8$ 。
n^4	$n \leq 125$	$n \leq 173$	$125^4 \approx 2.44 \times 10^8$, $173^4 \approx 8.96 \times 10^8$ 。
n^3	$n \leq 629$	$n \leq 965$	$629^3 \approx 2.49 \times 10^8$, $965^3 \approx 8.99 \times 10^8$ ；Floyd/DP 往往访存较重。
n^2	$n \approx 1.5 \times 10^4$	$n \approx 3.0 \times 10^4$	$15000^2 = 2.25 \times 10^8$ ； $20000^2 = 4 \times 10^8$ 可做； $30000^2 = 9 \times 10^8$ 为上限。存全表另算空间。
$n\sqrt{n}$	$n \approx 4.0 \times 10^5$	$n \approx 9.0 \times 10^5$	仅按简单操作量求根；分块/莫队常数与访存会把实际稳定规模压低。
$n \log_2 n$	$n \approx 1.0 \times 10^7$	$n \approx 3.5 \times 10^7$	仅按比较或简单更新计数；排序搬移及 BIT、线段树的随机访存应另行折算。
n	$n \leq 2.5 \times 10^8$	$n \leq 9 \times 10^8$	实际通常先受输入量和内存限制；这行只用于理论运算次数换算。
$K \cdot \text{状态}$	乘积 $\leq 2.5 \times 10^8$	乘积 $\leq 9 \times 10^8$	分层 DP/多参数算法直接把所有维度相乘，再按单次转移常数折算。

使用要求：先计算状态数、转移次数及测试组数的乘积，再与表中两条基准比较。 n^2 在 $n = 15000$ 时为稳定规模， $n = 20000$ 时为 4×10^8 次， $n = 30000$ 时达到上限线。时间估算通过后仍需单独核算空间；例如存储 n^2 条边或完整二维数组时，空间可能先超出限制。

12 *C++ 未定义行为与实现风险

种类	风险	错误示例	后果	修正方式
UB	有符号溢出	<code>int x=2000000000; x+=x;</code>	可变负；优化器可删分支	先提升到足够宽类型或除法预判
UB	乘后才转宽	<code>ll x=a*b; (两者为 int)</code>	乘法已在 int 溢出	<code>ll x=1LL*a*b;</code>
UB	非法移位量	<code>1ULL<<k, k < 0 或 k ≥ 64</code>	移位量越界	移位前检查； $k = 64$ 特判
UB	有符号左移	负数左移或结果不满足标准条件	任意结果	位掩码用无符号类型并检查位宽
UB	数组/容器越界	<code>int a[10]; a[10]=0;</code>	覆盖内存或段错误	显式检查；调试时用 <code>at()</code>
UB	除零 / 模零	<code>int x=a%b; (b = 0)</code>	可崩溃或任意行为	运算前检查分母/模数
UB	读未初始化值	<code>int x; if(x){...}</code>	结果不可预测	定义时初始化；不可达 DP 用 INF
UB	悬空引用/指针	返回局部变量地址；扩容后用元素引用	偶发 WA/RE	不返回局部地址；扩容前 <code>reserve</code> 或重取引用
UB	失效迭代器	<code>v.erase(it); ++it;</code>	跳过或崩溃	<code>it=v.erase(it)</code>
UB	非严格弱序	比较器返回 <code>a<=b</code>	<code>sort</code> 行为未定义	相等时必须两向均为 <code>false</code>
UB	未定求值顺序	<code>arr[i]=i++; (C++14)</code>	读写 <code>i</code> 未排序	拆行并保存旧值
UB	非虚析构删除	经基类指针删除派生对象	析构不完整或任意行为	需要多态删除时基类析构为 <code>virtual</code>
实现定义	负数右移	<code>int y=(-3)>>1;</code>	结果依赖实现	不以该操作代替除法；先明确编译器语义
资源	递归爆栈	链上 DFS 深度 10^5+	常见 SIGSEGV	显式栈或确认并调整栈限制
资源	无限递归	线段树区间不缩小	栈耗尽/超时	保证叶子终止且子区间严格缩小
编译	版本不兼容	C++14 使用 <code>std::gcd</code>	CE	自行实现 <code>gcd</code> ；不得使用 C++17 特性
提交	文件重定向错误	<code>freopen</code> 文件名、扩展名、模式或输入输出流不符	无法读入或写出答案	按题面和比赛公告逐项核对文件名、 <code>"r"/"w"</code> 及 <code>stdin/stdout</code>
提交	重定向未生效	<code>freopen</code> 仍被注释，或使用本机绝对路径	本地测试通过但评测无法读写	提交前在独立目录按正式文件名复测
编译	入口或预处理错误	<code>main</code> 、 <code>include</code> 拼写错误，括号缺失或源码重复粘贴	CE	使用正式编译命令重新编译最终文件，不以编辑器语法高亮代替编译
提交	文件格式错误	将 <code>.c</code> 、 <code>.pas</code> 或其他文件作为 C++ 源文件提交	编译失败或语言识别错误	核对题号、源文件名、扩展名和提交语言
输出	调试或固定答案残留	额外输出日志、样例特判、随机答案或固定字符串	WA	只保留题目规定的输出，并在样例外数据上复测
环境	非标准环境依赖	使用 <code>windows.h</code> 、 <code>system()</code> 或本机专用命令	CE、RE 或行为不一致	仅使用竞赛允许的标准库、编译选项和系统接口
源码	无关大段内容	源码中保留大样例、超大静态答案表或重复代码	文件超限、CE 或审查困难	删除与算法无关的内容，并核对最终源码大小
数值	浮点判等	<code>if(a==b)</code>	边界误判	按误差模型用 <code>eps</code> 或改整数
复杂度	哈希退化	<code>unordered_map</code> 出现严重冲突	单次最坏 $O(n)$	需要复杂度保证时使用排序、 <code>map</code> 或数组
概率	哈希碰撞	单哈希相等即判原串相等	小概率 WA	双哈希/强校验/确定性结构
类型	<code>vector<bool></code>	<code>vector<bool> v(1); auto x=v[0];</code>	<code>x</code> 是代理对象	需普通引用语义时用 <code>vector<char></code>

种类	风险	错误示例	后果	修正方式
类型	有符号/无符号混比	<code>for(int i=n-1;i<v.size();--i)</code>	负数转成大无符号数	下标统一为有符号宽类型并显式转换
复杂度	SPFA 退化	特殊构造反复入队	最坏 $O(VE)$	非负权优先 Dijkstra
逻辑	位运算优先级	<code>if(x&1==0)</code>	实为 <code>x&(1==0)</code>	写 <code>if((x&1)==0)</code>
逻辑	模减法为负	<code>(a-b)%MOD (a < b)</code>	输出负数	规范化到 $[0, \text{MOD})$
逻辑	线段树数组偏小	<code>vector<ll> tr(2*n)</code>	非满二叉规模时越界	递归版保守开 $4*n$
逻辑	多测不清空	全局数组/并查集复用	WA/RE	按本次实际范围重置
逻辑	混用整行输入	<code>cin>>n; getline(cin,s);</code>	读到残留换行	先用 <code>cin.ignore(...)</code> 清除换行, 再调用 <code>getline</code>

分类说明：UB 允许编译器作任意处理；实现定义行为由实现文档规定；未指定行为只限定若干可能结果。CE、资源耗尽、精度错误、复杂度错误和逻辑错误不属于 UB。提交与文件检查项根据 常见错误文档表.md 中的实际代码问题归纳。

13 * 对拍与差分测试

对拍由参考程序、待测程序、数据生成器和结果比较脚本组成。参考程序通常复杂度较高，生成数据的规模必须保证参考程序能够在限定时间内完成。

1. **brute.cpp**：采用枚举、回溯或全排列等易于验证的参考算法；数据规模应与其复杂度相符，常取 $n \leq 10 \sim 15$ 。
2. **sol.cpp**：与提交版本一致的待测程序。
3. **gen.cpp**：生成合法的小规模数据，使用 mt19937 和固定种子；同时覆盖链、全相等、全负、零边及最大值等边界。

编译 (Linux & Windows 通用): `g++ -std=c++14 -O2 -o brute brute.cpp (同理 sol、gen)`。Windows 下可执行文件自动加 `.exe` 后缀。

Linux / WSL / macOS (bash)

```
#!/bin/bash
for i in $(seq 1 1000); do
  ./gen "$i" > in.txt
  ./brute < in.txt > ans.txt
  timeout 5 ./sol < in.txt > out.txt
  if ! diff -q ans.txt out.txt > /dev/null; then
    echo "WA on test $i"; cp in.txt hack.in; exit 1
  fi
done
echo "All passed"
```

Windows (cmd)

```
@echo off
for /L %%i in (1,1,1000) do (
  gen.exe %%i > in.txt
  brute.exe < in.txt > ans.txt
  sol.exe < in.txt > out.txt
  fc ans.txt out.txt > nul
  if errorlevel 1 (
    echo WA on test %%i
    copy in.txt hack.in
    exit /b 1
  )
)
echo All passed
```

Windows 的 cmd 没有与 GNU `timeout` 等价的内置命令。需要限制单次运行时间时，可在 WSL 中使用 bash 脚本，或使用 PowerShell 的 `Start-Job` 与 `Wait-Job`。

边界数据应包括最小与最大规模、全等、全 0、全 1、全正、全负、链或菊花等退化树、零权边、环、相等关键字、无解输入及数值上界。

掌握要求：能够在限定时间内完成对拍框架；固定并记录随机种子；出现差异时保存失败数据，并分别排查生成器、参考程序、待测程序和比较方式。

14 *NP、NP 完全与 NP-hard 识别

本节用于识别一般问题的复杂性类别。NP 是判定问题类；NP 完全（NPC）问题同时属于 NP 和 NP-hard；优化问题通常称为 NP-hard，而不直接称为 NPC。题目给出小规模、小值域、树、DAG、二分图或固定参数时，应明确说明所利用的特殊结构。

一般问题	分类口径	处理原则
SAT/3-SAT、团、独立集、点覆盖、三染色、哈密顿路/回路	对应判定版为 NPC	$n \leq 20$ 时状压/枚举；图为树、DAG、二分图等时可能降为 P；先确认特殊结构。
子集和、划分、0/1 背包	判定版 NPC，但属弱 NP 完全的经典代表	$O(nS)$ 或 $O(nW)$ 是伪多项式：对数值大小多项式，不是对输入位数多项式；值域小仍可通过。
旅行商、最长简单路、最小染色、最大团、最小集合覆盖、Steiner 树	优化版 NP-hard	小 n 可使用 $O(n^22^n)$ 等精确 DP 或搜索；近似、随机和剪枝算法应注明正确性及最坏复杂度。
精确覆盖、支配集、反馈点集、3-Partition、一般整数规划	判定版多为 NPC；优化版 NP-hard	关注元素数、参数 k 、值域与结构；3-Partition 是强 NP 完全的警示，不能指望普通伪多项式 DP 消掉困难。
2-SAT、二分图匹配、最大流/最小割、MST、最短路、欧拉路、DAG 最长路	这些经典版本在 P	分类只适用于表中版本；3-SAT、一般图三染色和一般图最长简单路属于不同问题。

完整定义、经典列表、特殊结构和可用精确算法见 CSP-S-经典 NP-NPC-NP-hard 问题清单.md。识别要求：看到一个疑似困难问题时，先写清一般版本、判定/优化版本、输入编码和题目额外约束，再决定用指数算法、伪多项式 DP、参数化分支还是可证明的特殊结构算法。

15 能力缺口与复习重点

- 复习时应重点确认以下能力：
 1. **复杂度反推**：根据 $n \leq 10^5, m \leq 2 \times 10^5, K \leq 50$ 等多参数，计算 $O(K(n+m))$ 与 $O(n^2)$ 的实际操作量。
 2. **树与图建模**：将删边后的分量问题转化为重链跳跃与换根；将函数调用依赖转化为 DAG 拓扑序和反向贡献累积。
 3. **DP 状态设计**：使用差值状态、0/1 状态维及不可达值，严格区分「恰好」与「至多」。
 4. **部分分规划**：根据测试点的规模、固定参数和特殊结构确定可覆盖的分值并集。
 5. **数学推导**：完成方差到差分不变量的转化，并区分不同模数下的组合数算法及容斥边界。
 6. **对拍定位**：保存失败数据，并区分生成器、参考程序、待测程序和比较器错误。
- 线段树、树状数组、LCA、Tarjan、树上差分、状态压缩 DP 和树形 DP 常作为综合题的组成部分，不一定以独立模板形式出现。
- SCC、笛卡尔树、平衡树和 DP 常用优化是本文的重点；概率基本概念与平面图对偶作为拓展内容保留。
- 每项时间复杂度均应配套空间估算。一般图组合优化问题还需参照 CSP-S-经典 NP-NPC-NP-hard 问题清单.md，确认题目所利用的小参数或特殊结构。

创作声明与许可

除另有说明的第三方内容外，本文由 scmmm 原创。本文以 [Creative Commons 署名-非商业性使用 4.0 国际版 \(CC BY-NC 4.0\)](#) 发布。在注明原作者为 scmmm、保留本许可声明和许可证链接，并标明修改内容的前提下，允许非商业性转载、分享与修改。